

Semantic Icon Search in 4D Using AI and Vector Embeddings

By Karim Meghraoui, Technical Services Engineer, 4D Morocco

Technical Note 26-04

Table of Contents

Table of Contents	2
Abstract.....	3
Introduction	3
Concept Overview	3
Overview of the Application.....	4
Core Components.....	4
Text and Image	4
Embedding Generation and Storage	5
Similarity Computation.....	5
Ranked Result Presentation	6
Use Cases	6
Design Architecture	6
Requirements / Prerequisites.....	6
Technologies Used.....	7
OpenAI API Key Management.....	7
Problem and Proposed Solution	8
Application Workflow	10
High-level Workflow.....	10
Embedding Storage Strategy.....	11
Similarity Computation	11
Hybrid Search Strategy.....	12
Best Practices	12
Performance / Scalability.....	12
Conclusion	13

Abstract

Semantic search is an advanced information retrieval technique that improves result relevance by understanding the meaning of user queries rather than relying on exact keyword matching. This technical note presents a practical approach to implementing semantic search in a 4D application using vector embeddings generated via the 4D AI Kit and the OpenAI embedding API.

Introduction

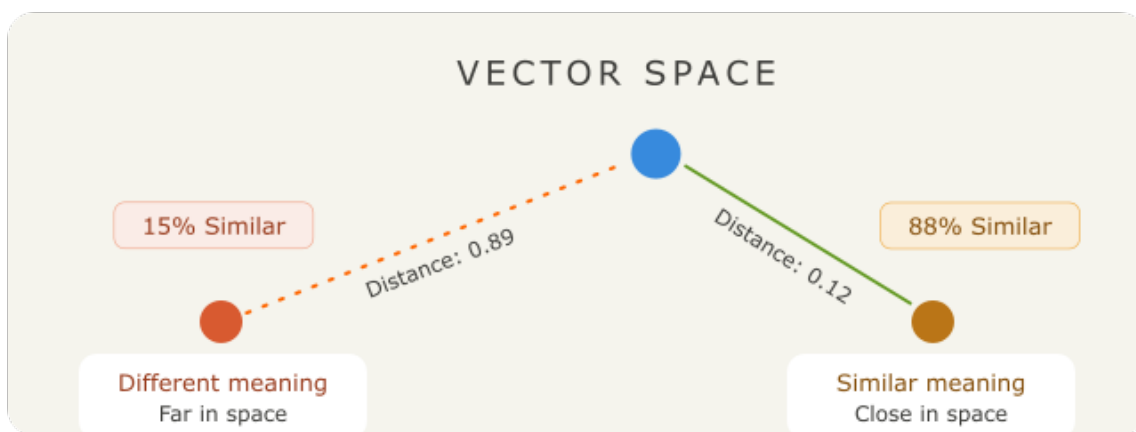
This document explains the fundamental concepts behind embeddings, vector similarity, and ranking mechanisms, and demonstrates how these concepts can be applied within a 4D environment using ORDA and the 4D.Vector data type. It also describes a hybrid search strategy combining dynamic embedding generation and precomputed vectors to optimize performance and reduce external API usage.

Through detailed explanations and annotated code examples, this technical note provides developers with the knowledge required to design and implement efficient, scalable, and user-friendly semantic search systems across a variety of use cases.

Concept Overview

Semantic search is a search technique that focuses on understanding the meaning and intent of a query instead of relying solely on keyword matching. It is based on embeddings, which are numerical representations of text generated by machine learning models.

When text is converted into an embedding, it becomes a vector of numerical values representing semantic features. Texts with similar meanings produce vectors that **are located close to each other in vector space**.

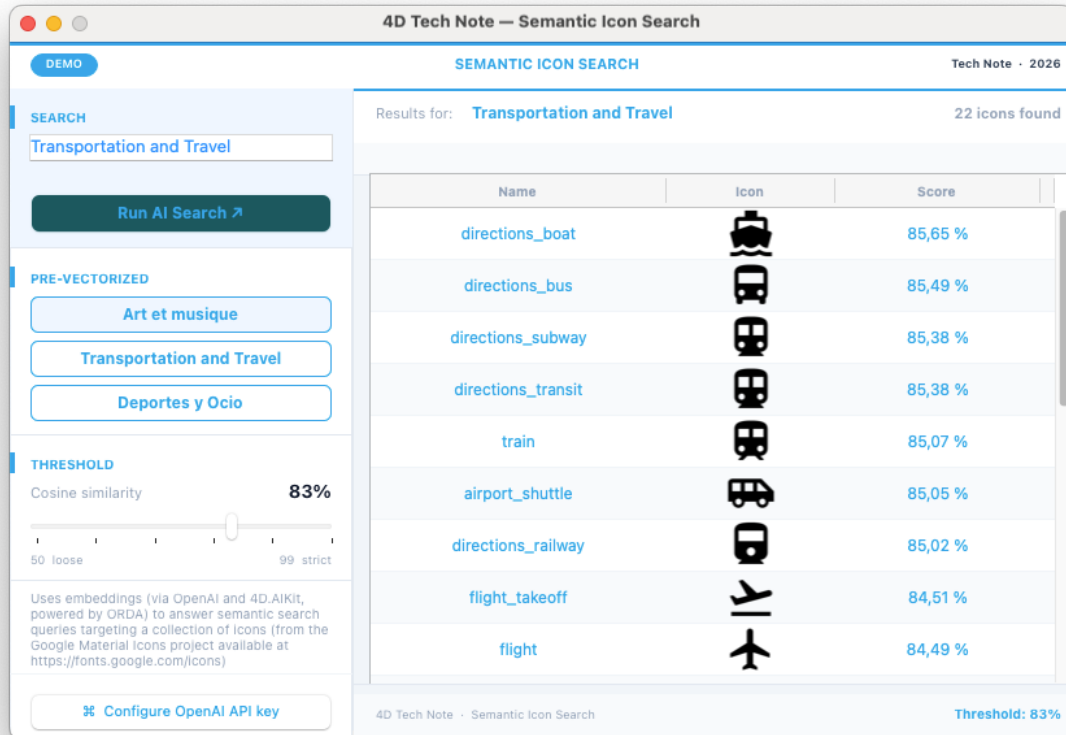


By comparing vectors, the system can determine how closely related two pieces of text are.

In the context of an icon library, semantic search enables users to retrieve icons based on conceptual similarity rather than exact metadata matches.

Overview of the Application

The sample application includes:



- A search input field where users can enter a query
- Three predefined keyword buttons with precomputed embeddings stored in the database
- A similarity threshold slider to adjust result precision
- A "Configure OpenAI API Key" button that must be used before running a search
- A results panel showing icon names, similarity scores, and images

When a search is executed, the system generates a vector embedding for the query and compares it with the stored embeddings for each icon. The most semantically relevant icons are returned and ranked by similarity.

Core Components

Text and Image

Each icon contains two types of information:

- Image data, which represents the visual icon displayed in the interface
- Textual metadata, such as the icon name and associated tags

The textual metadata is used to generate embeddings representing the semantic meaning of the icon.

Embedding Generation and Storage

Embeddings generated from icon metadata are stored directly in the database using the 4D.Vector data type.

Each icon entity therefore contains:

- Name
- Picture
- Tags
- Vector embedding

These representations are generated once during the preprocessing phase and then stored in the database for all subsequent queries.

This code illustrates how text is transformed into a vector and stored. The embedding is generated using an external model and then saved as part of the entity. This step is essential for performance, as it avoids repeated calls to the embedding service during search operations.

Example:

```
$embeddingResult:=$embeddingAPI.create($iconEntity.tags; $model)
If ($embeddingResult.vector#Null)
    $iconEntity.vector:=$embeddingResult.vector
    $iconEntity.save()
End if
```

When vectors are stored in the database, ORDA can perform quick and efficient similarity comparisons.

Similarity Computation

Once vectors are available, the system must determine how similar they are. This is done using cosine similarity, a mathematical measure that evaluates the angle between two vectors.

Instead of comparing text directly, the system compares their vector representations. If two vectors point in similar directions, their cosine similarity is high, indicating that the underlying texts are semantically related.

In the application, similarity is computed directly within the database query.

The following query compares a query vector with stored vectors, applies a similarity threshold filter, and sorts results in descending order of similarity:

```
$iconSelection:=ds.Icon.query("vector >= :1 order by vector desc"; \
    {vector: $embeddingResult.vector; metric: mk_cosine; \
    threshold: $thresholdPercentage/100}
```

The use of cosine similarity ensures that results are ranked according to semantic relevance. This demonstrates the power of ORDA queries applied to vector fields.

Ranked Result Presentation

After similarity has been computed, the system must present the results in a meaningful way. Each result is associated with a similarity score, which reflects how closely it matches the query.

The code below demonstrates how scores are calculated and attached to results:

```
For each ($iconEntity; $iconSelection)
    $resultWithScore:=New object()
    $resultWithScore.name:=$iconEntity.name
    $resultWithScore.picture:=$iconEntity.picture
    $resultWithScore.score:=100*($iconEntity.vector.cosineSimilarity(\
        $embeddingResult.vector))
    $results.push($resultWithScore)
End for each
```

This step transforms raw query results into structured objects that include both the content and a readable similarity score. Ranking ensures that the most relevant items appear first, improving usability and search efficiency.

A 4D ListBox is then used to easily present all search results in a useable interface. Sorting by name or by similarity score is directly possible with no additional code through a ListBox.

Use Cases

Although this example focuses on an icon library, semantic search can be applied to many other domains, including:

- document repositories
- knowledge bases
- media libraries
- product catalogs
- image collections

In all these scenarios, semantic search improves the ability to find relevant items even when users do not know the exact keywords used in the metadata.

Design Architecture

Requirements / Prerequisites

To implement semantic search in this application, the following prerequisites are required:

- A 4D project using ORDA for data access
- Support for the 4D.Vector data type
- The 4D AI Kit component added to the project dependencies

- An OpenAI API key to generate embeddings
- A dataset of icons with associated metadata (the [Material Design Icons project](https://fonts.google.com/icons) by Google was used for the example application in this technote”
<https://fonts.google.com/icons>”)

The 4D AI Kit component must be added to the project dependencies before using the embedding API.

Technologies Used

The implementation relies on several technologies provided by the 4D platform.

- **ORDA** is used to access entities and perform database queries.
- **4D.Vector** is used to store embedding vectors directly in the database.
- **4D AI Kit** provides a simplified interface to interact with OpenAI services.

Managing OpenAI API Key: The OpenAPI key is handled in a simple and safe way within the application: the storage is non-persistent (hence avoiding any accidental leak or undesired propagation via a data file or a config file). The key must be provided each time the demo application is started. This approach is safe and easily manageable for a demo application, but for more complex scenarios that necessitate persistence, proper security practices must be applied to ensure the safety of the key.

OpenAI API Key Management

Users can enter or update their key via a dialog, which is automatically pre-filled with any previously saved value. This ensures that the key can be reused throughout the application without modifying the code each time.

Example of how the key is handled in 4D:

```
var $key : Text
$key:=Request("Enter your OpenAI Key: "; GetOpenAIKey())
If (OK=1)
    SetOpenAIKey($key)
End if
```

GetOpenAIKey() reads the key from storage if it exists:

```
Use (Storage)
If (Storage.openAIKey#Null)
    Use (Storage.openAIKey)
    $key:=Storage.openAIKey.value
End use
End if
End use
```

SetOpenAIKey(\$key) ensures the storage object exists and saves the key:

```
Use (Storage)
If (Storage.openAIKey=Null)
    Storage.openAIKey:=New shared object()
```

```
End if
Use (Storage.openAIKey)
  Storage.openAIKey.value:=$key
End use
End Use
```

This mechanism leverages 4D Storage and shared objects to provide a secure and reusable way to manage OpenAI credentials with no persistence, enabling seamless access to AI Kit services such as the embedding API.

Problem and Proposed Solution

Traditional icon search systems rely on exact keyword matching. Icons are associated with metadata such as names or tags, and results are returned only when the query contains the same words.

This approach has limitations. For example, an icon tagged with: robot automation machine might not appear when the user searches for: automated system

even though both expressions describe similar concepts.

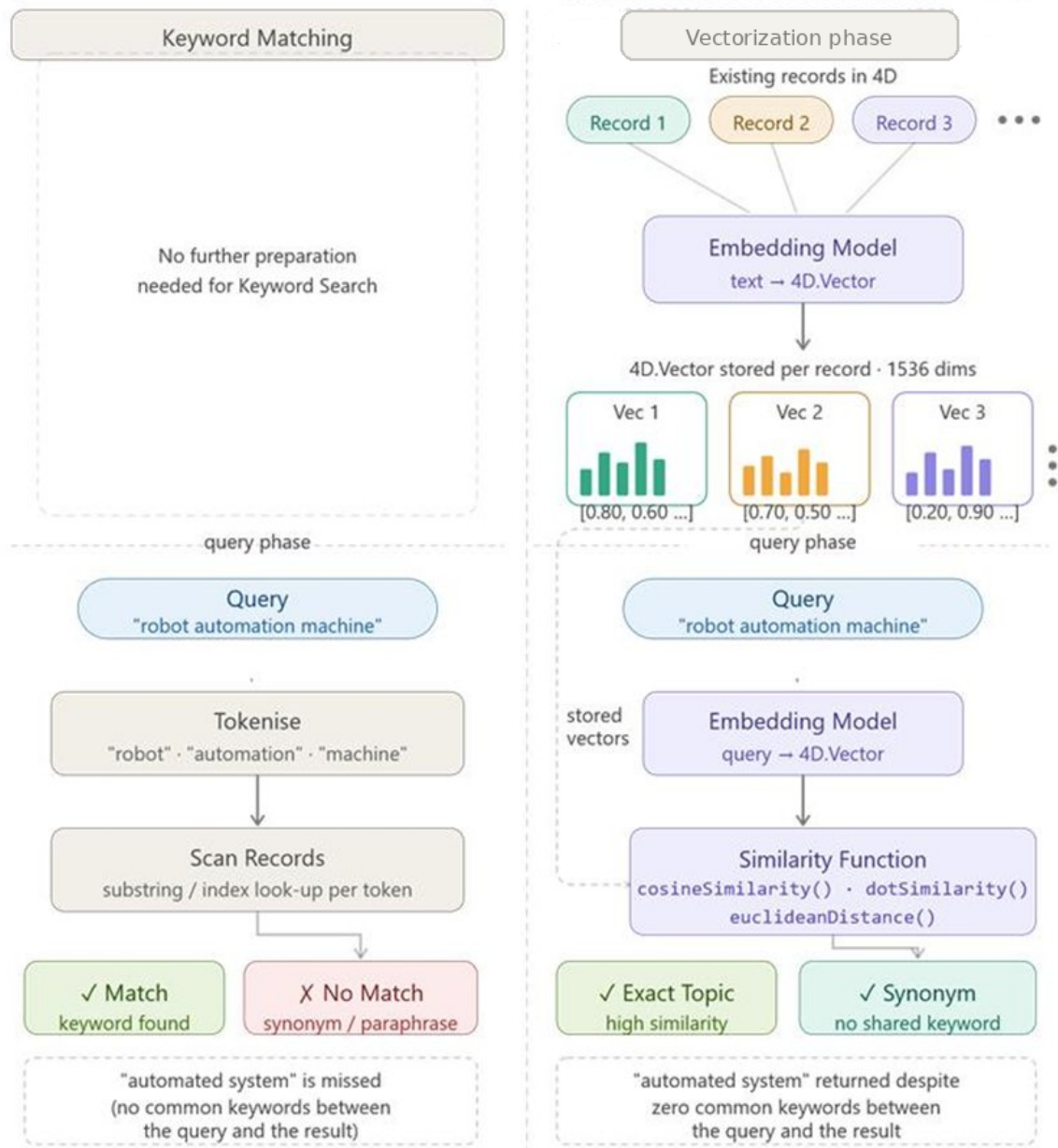
To solve this limitation, the application uses semantic search based on vector embeddings.

Each time the user submits a search prompt, it is processed by an embedding model, which converts each text input into a numerical vector representation. These vectors capture the semantic meaning of the prompts across many dimensions.

The left side of the diagram below shows keyword matching: the query is split into individual words, and the system searches for exact matches in the data. This approach is simple and fast, but it only works when the same words are used, so it misses synonyms or paraphrases.

The right side shows semantic search: during an indexing phase, all records are converted into vectors using an embedding model. At query time, the query is also turned into a vector and compared to stored vectors using a similarity function. This allows the system to find results based on meaning, even if the exact words are different.

Keyword Matching vs Semantic Search in 4D

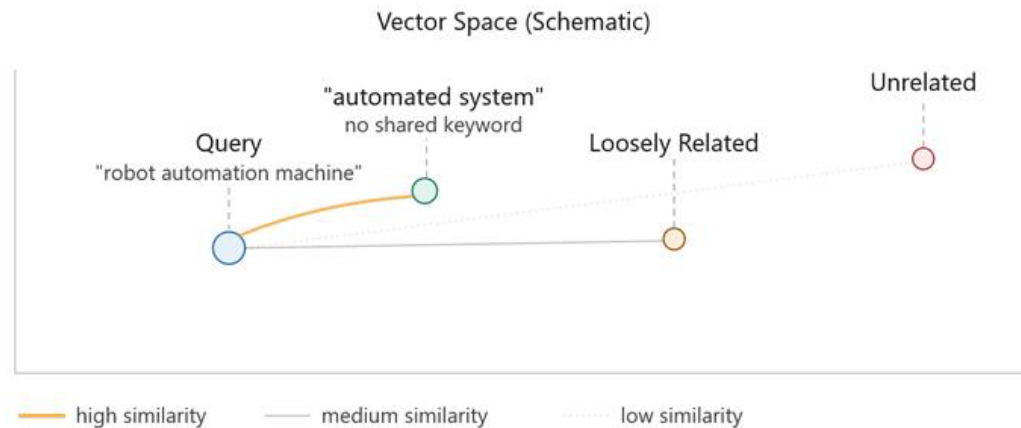


In this approach:

- Textual metadata describing each icon is converted into a vector embedding
- User search queries are also converted into vectors using the same model
- The system compares these vectors using similarity metrics

The diagram below shows how words or phrases are represented in a vector space based on their semantic similarity. The query "robot automation machine" is close to the phrase "automated system" (high similarity), even without shared keywords, because they have a related meaning. Loosely related terms are farther away with low similarity, and unrelated

terms are the most distant, indicating no semantic connection. This illustrates how embedding models capture meaning beyond exact word matches.



Because embedding captures semantic relationships between words, the system can retrieve icons that are conceptually related to the query, even when different wording or language are used.

Application Workflow

High-level Workflow

The semantic search workflow consists of two **main phases**.

Phase 1 – Data Import and Vectorization

Icons are imported from a 4D resources folder and stored in the database with their metadata.

In the resources folder within the project, a subset of icons (in PNG format) and a list of tags (in a JSON file) were extracted from the [Material Design Icons](https://fonts.google.com/icons) project (available at <https://fonts.google.com/icons>).

The following code creates a new Icon entity (in the Icons table) for each icon file found in the resources directory.

Each icon entity has a name (of type Text), a graphic representation (of type Picture), a list of tags (space separated, and stored as a single string), and an associated vector (of type 4D.Vector).

The name, tags, and picture data are first set for each icon, while the vector field is left empty (null).

```
$iconEntity:=ds.Icon.new()  
$iconEntity.name:=$iconFile.name  
$iconEntity.picture:=$iconPicture  
$iconEntity.tags:=$iconTags[$iconFile.name].join(" ")  
$iconEntity.vector:=Null  
$iconEntity.save()
```

Phase 2 – Embedding Generation

Each icon's tags are converted into an embedding vector using OpenAI through the 4D AI Kit.

Once all data is stored in the data file, all icon entities are iterated to compute a vector embedding from their associated tags. The OpenAI API is used through 4D.AIKit to access the embedding service, using the “text-embedding-ada-002” model. This model converts text input into a numerical representation (vectors) that captures the semantic meaning of the provided text.

```
var $model : Text  
$model:="text-embedding-ada-002"  
var $embeddingAPI : cs.AIKit.OpenAIEmbeddingsAPI  
$embeddingAPI:=GetOpenAIEmbeddingAPI()  
$embeddingResult:=$embeddingAPI.create($iconEntity.tags; $model)
```

The resulting vector is then stored in the icon entity.

This process is typically **executed once during vectorization**.

Embedding Storage Strategy

Embeddings are generated during the vectorization phase and stored directly in the database. ORDA now supports fields of type 4D.Vector in entities and allows fast and optimized vector queries on vector fields.

This approach ensures that search operations remain fast because vectors do not need to be recomputed for each query.

Only new icons or modified metadata require embedding regeneration.

Similarity Computation

When a user enters a search query, the text is converted into an embedding vector using the same model used during indexing.

The application then performs a similarity query comparing this vector with stored icon vectors. Using 4D, this is easily performed via a single call to .query() function, with the appropriate parameters for vector search.

The results are sorted according to similarity, and a threshold value can be used to filter out less relevant results.

Hybrid Search Strategy

In the sample application, two types of searches are demonstrated.

1. Dynamic Search via OpenAI

When a user submits a custom prompt, the system generates a vector embedding via OpenAI and compares it against precomputed embeddings of the icon dataset to perform a similarity search.

This approach enables flexible and intuitive querying using natural language, including multilingual input when supported, without relying on predefined keywords. The system retrieves icons with the closest semantic meaning to the query and ranks results by similarity score, ensuring that the most relevant matches are returned first.

2. Local Search Using Precomputed Vectors

The application contains three predefined search buttons, each associated with a search prompt.

The embeddings for these prompts are generated once and stored in the database.

When the user clicks one of these buttons, the application directly uses the stored vector to perform a similarity query without calling the OpenAI API again.

This demonstrates that semantic search can be performed:

- with an external AI service (dynamic queries)
- or locally using previously generated vectors

This hybrid approach helps reduce API calls while maintaining fast semantic search capabilities.

Best Practices

The following principles define a concise approach to building an efficient and scalable semantic search system:

- **Vectorize Once, Use Many:** Generate and store embeddings during the vectorization phase to avoid repeated API calls and improve performance.
- **Leverage Semantic Similarity:** Use embeddings to capture conceptual relationships between metadata and user queries.
- **Hybrid Search Strategy:** Combine dynamic search via OpenAI with local precomputed vectors to reduce API usage and speed up queries.
- **Extendable Architecture:** This approach can be applied to other domains: documents, media, product catalogs, knowledge bases, etc.

Performance / Scalability

Performance optimization is important when implementing semantic search with large datasets.

The main recommendation is **to generate embeddings during the vectorization phase** and store embeddings in the database, avoiding repeated calls to the embedding service during search operations.

Another optimization strategy is **to reuse stored vectors** for predefined searches, as demonstrated with the keyword buttons in the sample application.

Conclusion

Semantic search using vector embeddings significantly enhances the usability of icon libraries in 4D applications.

By converting both icon metadata and user queries into vector representations, the system can identify conceptual similarities and retrieve icons based on meaning rather than exact keywords.

Using **ORDA, 4D.Vector, and 4D AI Kit component**, developers can easily integrate modern AI-powered search capabilities into their 4D applications.

This architecture can be extended beyond icon libraries to many other domains where intelligent information retrieval is required, such as document search, media management, and knowledge base exploration.